🧪 **LABORATÓRIO**

O cenário

Um time mantém um **classificador de texto** que rotula avaliações de produto como *positiva*, *negativa* ou *neutra*. Hoje, o modelo é retreinado manualmente uma vez por mês e o deploy é feito por um engenheiro que copia o arquivo do modelo para o servidor.

Em grupos, vocês vão **decidir** os critérios do pipeline de CI/CD que substitui esse processo manual — e depois fazê-lo **rodar de verdade**.

🧪 **LABORATÓRIO · DUAS PARTES**

Decidir, depois fazer rodar

Parte 1 · Desenho (≈12 min)

- As **sete etapas** aplicadas ao classificador
- Para cada uma: o que valida, um **critério concreto** e o que acontece se falhar
- Cada grupo escolhe uma das três propostas

Parte 2 · Prática (demo de 5 + ≈22 min, depois do intervalo)

- O pipeline **de verdade**, no repositório-base da disciplina
- Ver o *model gate* barrar um lote ruim
- Deploy de uma função na **AWS**

O desenho da Parte 1 é a **especificação** do que vocês vão verificar na Parte 2.

Três propostas — cada grupo escolhe uma

Mesmo cenário (o classificador de avaliações), três profundidades diferentes. Escolham pela experiência do grupo — todas terminam no mesmo diagrama de sete etapas.

A · Pipeline mínimo viável

- Para quem ainda não montou uma esteira de CI/CD
- Foco: as **sete etapas**, cada uma com um critério concreto
- Vem com uma tabela-modelo parcialmente preenchida

B · O gate que decide

- Para quem já tem CI/CD de código no dia a dia
- Foco: **testes de dados** e o **model gate**
- Traz um lote novo de dados que "parece normal"

C · Do mensal ao contínuo

- Para quem já opera modelos em produção
- Foco: **observabilidade** que dispara o retreino
- Fecha o ciclo: produção realimenta o pipeline

Parte 1: ~12 minutos de trabalho em grupo, em salas do Teams. Guardem o diagrama: ele volta no trabalho final do case Quantum Commerce.

🧪 PROPOSTA A · PIPELINE MÍNIMO VIÁVEL

Uma linha por etapa, sem deixar critério vago

Completem a tabela para o classificador. As duas primeiras linhas são o exemplo do nível de detalhe esperado:

Etapa	O que valida	Critério de aprovação	Se falhar
Lint	Estilo e imports do código de treino	Zero erros no linter	Bloqueia o PR
Testes unitários	Limpeza de texto (acentos, emojis, URLs)	100% dos testes passam	Bloqueia o PR
Testes de dados	?	?	?
Build	?	?	?
Model gate	?	?	?
Deploy	?	?	?
Observabilidade	?	?	?

Pista: se o critério não cabe num `if` que uma máquina consegue avaliar, ele ainda não é um critério.

🧪 PROPOSTA B · O GATE QUE DECIDE

Chegou um lote novo — ele pode virar modelo?

O time recebeu 20 mil avaliações novas para o retreino do mês. O lote passa no schema, mas **30% dos textos vêm só com emojis** e a classe *neutra* caiu de 25% para 9% dos rótulos.

Testes de dados

- Que checagens de **distribuição** pegariam esse lote?
- Qual limite dispara bloqueio — e qual só gera alerta?

Model gate

- Candidato **vs.** produção: qual métrica e qual margem?
- Basta a média, ou cada classe precisa de um piso próprio?

Entregável: o diagrama das sete etapas, com as etapas 3 (dados) e 5 (gate) detalhadas — e a decisão: esse lote vira modelo ou não?

🧪 **PROPOSTA C · DO MENSAL AO CONTÍNUO**

Retreinar quando precisa, não quando o calendário manda

Em produção quase não há rótulo: ninguém diz ao sistema se a previsão estava certa. Desenhem como a **observabilidade** decide que é hora de retreinar — e como o modelo novo volta pelo mesmo pipeline.

Sinal em produção → **Gatilho de retreino** → **Pipeline (7 etapas)** → **Deploy ou rollback**

- Quais **três sinais** vocês monitoram sem rótulo? (pistas: distribuição das previsões, confiança média, tamanho/idioma do texto de entrada)
- Qual combinação de sinais **dispara** o retreino — e o que só alerta?
- Se o modelo novo piorar depois do deploy, qual é o **rollback**?

🧪 PARTE 2 · PREPARAÇÃO

Onde cada coisa roda

O quê	Onde	Custo
Código, CI e *secrets*	Repositório seu, criado a partir do **template** da disciplina	gratuito
Terminal e editor	**GitHub Codespaces** (abra no repositório)	gratuito (120 h-core/mês)
Deploy do modelo	**AWS Academy** Learner Lab — Lambda + ECR	crédito do Learner Lab

1. **Use this template** em `github.com/guilherme-argentino/aie-devops-labs` (repositório **público**) e abra um Codespace. Ele aparece como "disponível" antes de terminar: espere o terminal acabar de configurar (~8 min). **Já tem a cópia da Aula 1? Use essa mesma:** ela já traz tudo de hoje
2. No Learner Lab: **Start Lab** → **AWS Details** e copie as três credenciais para os *secrets* do repositório (`roteiros/setup-aws-academy.md`)

As credenciais do Learner Lab **expiram com a sessão de 4 horas**, e não conte com a função e a URL fora da sessão (com o lab encerrado a URL **pode dar erro 502**): a cada sessão, *Start Lab*, credenciais novas e, se preciso, o deploy de novo (1 a 2 min).

🧪 PARTE 2 · O PIPELINE RODA E O GATE DECIDE

Um lote que "parece normal"

```
git push                          # o CI dispara; acompanhe: gh run watch

DADOS=classificador/dados/lote-novo.csv \
  pytest classificador/tests/test_dados.py     # etapa 3
python classificador/treino.py \
  --dados classificador/dados/lote-novo.csv    # etapa 4
python classificador/model_gate.py \
  artefatos/metricas.json                       # etapa 5
```

Esperado: o **gate reprova** — F1 macro 0,73 (< 0,80) e a classe *neutra* em 0,58 (< 0,70).

Para discutir: o que barrou o lote primeiro, o teste de dados ou o gate? Qual regra bloqueia e qual só alerta? Mudem um limite e justifiquem.

🧪 PARTE 2 · DEPLOY NA AWS

Etapa 7 no AWS Academy — e o plano B

```
gh workflow run "Aula 2 · Deploy do classificador (AWS Lambda)"
```

O workflow refaz treino e gate, publica a imagem no **ECR**, cria uma **versão** da função Lambda, aponta o alias `producao` e testa a URL pública.

Plano B (credenciais expiradas ou serviço bloqueado no seu Learner Lab): a etapa 6, o *build* da imagem, já passou no CI. Rodem o mesmo contêiner localmente:

```
python classificador/treino.py
docker build -f classificador/Dockerfile -t classificador .
docker run -p 9000:8080 classificador
```